4.1. СТРУКТУРЫ GO ГЛУБЖЕ

growslice · Swiss Table · HashTrieMap · ABI интерфейсов

01 / SLICES

nextslicecap
аллокатор + memmove

02 / MAP

H1 / H2, битовые маски
probing и split

03 / SYNC.MAP

HashTrieMap
атомики + локи

04 / INTERFACE

Type / ITab + Data
assertion, dispatch

Граница: семантика языка / решения компилятора / детали Go 1.27.x

Четыре конструкции — четыре разных алгоритмических задачи

append	Как выбирается реальный cap?
map	Как H2 и маска ищут слот?
sync.Map	Где атомики, где лок?
interface	Как получить Type / ITab и вызов?

Не перечитываем GC, allocator, планировщик, sudog и MESI: используем их как известные механизмы.

Срезы: от append до физического массива

Как runtime выбирает емкость и копирует данные?

nextslicecap

roundupsize

pointer / noscan

memmove

Исходник: 4.1 - Go structures deeper.md

append: известная модель заканчивается на границе cap

МЕХАНИЗМ

```
старый массив
↓
есть свободная ячейка
↓
записать x
↓
len++
```

ПОЧЕМУ ТАК

- Только переполнение cap заставляет append искать новый базовый массив.
- рантайму новый базовый массив не нужен.

Кто вызывает growslice

МЕХАНИЗМ

```
append
↓
проверка емкости
↓
места нет
↓
runtime.growslice
```

ПОЧЕМУ ТАК

- Проверку cap строит компилятор; growslice — медленный путь.
- Когда не хватает, появляется путь через внутреннюю функцию рантайма:
- Здесь считаем, что мы уже попали внутрь growslice.

Какие данные получает growslice

МЕХАНИЗМ

```
oldPtr   → старый базовый массив
newLen   → новая длина
oldCap   → старая емкость
num      → сколько элементов добавляем
et       → описание типа в рантайме типа элемента
```

ПОЧЕМУ ТАК

- Новый размер зависит от oldCap, newLen и описания типа T.
- Потому что одного числа элементов недостаточно.
- Текущая сигнатура концептуально выглядит так:

Сначала нужно выбрать новый cap

МЕХАНИЗМ

```
newLen
+
oldCap
↓
nextslicecap
↓
кандидат newCap
```

ПОЧЕМУ ТАК

- nextslicecap выбирает кандидата, а не окончательный размер аллокации.
- Для маленьких срезов она долго выглядит правдоподобно.

Первый случай nextslicecap: запрос слишком большой

МЕХАНИЗМ

```
oldCap = 100
newLen = 500
2 × oldCap = 200  <  500

newCap = 500
```

ПОЧЕМУ ТАК

- Если newLen превышает 2 × oldCap, промежуточное удвоение бесполезно.
- nextslicecap сразу выбирает newLen — никакого многоступенчатого роста.

Маленькие срезы: рост примерно в два раза

МЕХАНИЗМ

```
threshold = 256
oldCap = 64
newLen = 65

newCap = 128
```

ПОЧЕМУ ТАК

- При oldCap < 256 и умеренном приросте кандидат обычно равен 2 × oldCap.
- 256 — порог реализации Go 1.27, не гарантия языка.

Большие срезы: переход к более плавному росту

МЕХАНИЗМ

```
newCap += (newCap + 3*threshold) / 4

threshold = 256

при больших cap:
newCap ≈ newCap × 1.25
```

ПОЧЕМУ ТАК

- Дополнительное слагаемое дает плавный переход от ×2 к примерно ×1.25.
- Формула выбирает кандидат; реальный cap еще изменит округление аллокатора.

Разберем рост руками

МЕХАНИЗМ

```
oldCap = 512
newLen = 513
threshold = 256

newCap = 512 + (512 + 768) / 4
       = 512 + 320
       = 832

832 / 512 = 1.625
```

ПОЧЕМУ ТАК

- Коэффициент роста при 512 элементах равен 1.625, а не 1.25.
- При больших значениях влияние 3 × threshold уменьшается.

Зачем вообще уменьшать коэффициент роста

МЕХАНИЗМ

```
Больший коэффициент
  → меньше перевыделений
  → больше лишней памяти

Меньший коэффициент
  → меньше запаса
  → больше возможных ростов
```

ПОЧЕМУ ТАК

- Стратегия роста балансирует число аллокаций и избыточную емкость.
- При росте одновременно существуют старый и новый массивы; копирование тоже стоит ресурсов.

nextslicecap еще не определил фактический cap

МЕХАНИЗМ

```
cap результата = 1000
```

ПОЧЕМУ ТАК

- Логический newCap и фактический cap могут различаться.
- Потому что аллокатор не выдает память в единицах «элементы T».
- Вот важный слой, который обычно пропускают.

Из элементов в байты

МЕХАНИЗМ

```
запрошенный размер в байтах
↓
roundupsize
↓
реально выделенный размер
```

ПОЧЕМУ ТАК

- Аллокатор работает с байтами, а не с количеством элементов T.
- Почему roundupsize вообще нужен, мы уже разбирали в GC deep: size classes, выравнивание и классы аллокаций.
- Здесь нас интересует другое следствие.

Аллокатор может изменить фактический cap

МЕХАНИЗМ

```
candidate cap = 1000 элементов
sizeof(T) = 24 B

1000 × 24 = 24 000 B
roundupsize → 24 576 B

24 576 / 24 = 1024
actual cap = 1024
```

ПОЧЕМУ ТАК

- nextslicecap возвращает логическую емкость-кандидат.
- Аллокатор округляет байты до подходящего размера, поэтому фактический cap может быть больше.

Почему в growslice несколько вариантов расчета размера

МЕХАНИЗМ

```
sizeof(T) == 1
sizeof(T) == размер указателя
sizeof(T) — степень двойки
общий случай
```

ПОЧЕМУ ТАК

- Специализация sizeof(T) экономит арифметику на горячем пути.
- Потому что горячий путь не хочет делать дорогую арифметику, если ее можно упростить.
- Текущий код специально выделяет несколько случаев:

Проверка переполнения — часть growslice

МЕХАНИЗМ

```
newCap × sizeof(T)
          ↓
overflow?  /  capmem > maxAlloc?
          ↓
паника или безопасная аллокация
```

ПОЧЕМУ ТАК

- Без проверки переполнения размер выделения мог бы оказаться меньше требуемого.
- Проверка корректности нужна при переходе из количества элементов к байтам.

Теперь нужно получить новую память

МЕХАНИЗМ

```
mallocgc
→ mcache
→ mspan
→ mcentral
→ mheap
→ page allocator
```

ПОЧЕМУ ТАК

- После расчета байтов growslice обращается к уже изученному mallocgc.
- growslice запросил новый базовый массив у известного нам аллокатора.

[]byte и []*Node: почему наличие указателей меняет путь

МЕХАНИЗМ

```
[]byte
→ pointerless / noscan
→ возможно меньше очистки

[]*Node
→ pointerful / scan
→ нулевое состояние
→ пакетный барьер при копировании
```

ПОЧЕМУ ТАК

- Одинаковая семантика append не означает идентичный медленный путь.
- Рантайм учитывает sizeof(T) и наличие Go-указателей.

Срез без указателей: можно не очищать то, что сейчас перезапишет append

МЕХАНИЗМ

```
новый буфер []byte

[0, oldLen)
  → скопировать старые bytes
[oldLen, newLen)
  → append скоро перезапишет
[newLen, newCap)
  → нужно обнулить хвост
```

ПОЧЕМУ ТАК

- Для памяти без указателей область будущих записей append можно временно не очищать.
- Неиспользуемый хвост должен сохранять нулевую семантику.

Срез с указателями: неинициализированные указатели недопустимы

МЕХАНИЗМ

```
новый [] *Node
        ↓
память содержит Go-указатели
        ↓
нулевая инициализация
        ↓
GC не увидит случайные старые биты
```

ПОЧЕМУ ТАК

- GC может сканировать pointerful-массив до того, как новые элементы будут записаны.
- Поэтому нельзя оставить потенциально мусорные указатели.

Пакетный барьер записи: причину мы уже знаем

МЕХАНИЗМ

```
старый базовый массив
↓
много элементов с указателями
↓
новый базовый массив
```

ПОЧЕМУ ТАК

- Пакетный барьер записи позволяет безопасно копировать много указателей при concurrent GC.
- Поэтому рантайм имеет пакетные механизмы для подготовки барьера перед массовым memmove.
- Копировать каждый указатель как полностью независимую пользовательскую запись было бы невыгодно.

Старые элементы копируются через memmove

МЕХАНИЗМ

```
oldPtr
  │
  └── memmove(oldLen × sizeof(T))
                    ↓
                 newPtr

копируем len, не cap
```

ПОЧЕМУ ТАК

- growslice переносит только логически существующие элементы.
- Байты за oldLen не обязаны сохраняться при перевыделении.

growslice не записывает добавляемые элементы

МЕХАНИЗМ

```
growslice
├── подготовить новый массив
├── скопировать [0, oldLen)
└── вернуть новый descriptor

compiler / caller
└── записать [oldLen, newLen)
```

ПОЧЕМУ ТАК

- Добавляемые значения записывает сгенерированный код append.
- Это позволяет оптимизировать очистку области будущих элементов.

Особый случай: тип нулевого размера

МЕХАНИЗМ

```
type Empty struct{}

sizeof(Empty) == 0

[]Empty  →  нет отдельных bytes
         →  runtime.zerobase
```

ПОЧЕМУ ТАК

- Количество элементов и объем физической памяти — разные величины.
- Для нулевого размера есть специальный путь рантайма.

Полный медленный путь append

МЕХАНИЗМ

```
append
↓
newLen > oldCap
↓
growslice
↓
nextslicecap
↓
кандидат newCap
↓
newCap × sizeof(T)
↓
roundupsize
↓
фактический newCap
…
```

ПОЧЕМУ ТАК

- Медленный append = расчет cap → аллокация → memmove → запись добавленного.
- имеет законченную механическую модель.

Что происходит под нагрузкой со slice

МЕХАНИЗМ

```
пропускная способность памяти ↑
скорость аллокаций ↑
работа GC ↑
CPU ↑
p99 ↑
```

ПОЧЕМУ ТАК

- Цена роста slice — аллокации, копирование, пропускная способность памяти и GC.
- Как часто именно этот append пересекает границу cap и сколько данных при этом копирует?
- Получаем уже гигабайты памяти, проходящие через memmove.

Диагностика роста slice

МЕХАНИЗМ

```
профиль CPU
+
alloc_space
+
benchmem
```

ПОЧЕМУ ТАК

- Диагностировать нужно частоту и размер роста, а не слово append.
- полезен только если expected реалистичен.
- Но оптимизация должна быть измеримой.

Map: Swiss Tables до битовых операций

Как восемь управляющих байтов становятся маской кандидатов?

H1/H2

matchH2

probeSeq

directory + split

Исходник: 4.1 - Go structures deeper.md

Переходим к map: предыдущая модель заканчивается на слове «группа»

МЕХАНИЗМ

```
ключ
↓
хеш
↓
H1 / H2
↓
группа Swiss Table
↓
ячейки-кандидаты
↓
сравнение ключей
```

ПОЧЕМУ ТАК

- Swiss Table сначала фильтрует слоты компактными метаданными, потом сравнивает ключи.
- Вот здесь начинается настоящий Swiss Table.
- Как одна группа проверяет восемь H2 практически одновременно?

Текущая Go map: несколько важных фактов

МЕХАНИЗМ

```
Map
  ↓
каталог (directory)
  ↓
таблица (table)
  ↓
группа (8 slots)
  ↓
ctrl word + key/value
```

ПОЧЕМУ ТАК

- Хеш: H1 — верхние биты для навигации, H2 — нижние 7 бит.
- Малая map может обойтись одной группой без каталога.

Одна группа — восемь логических ячеек

0	1	2	3	4	5	6	7
12	35	80	35	7F	FE	22	35

Ищем H2 = 35. Кандидаты — ячейки 1, 3 и 7.

control bytes → H2 filter → mask → сравнение реальных ключей

Одна группа Swiss Table = 8 слотов и 8 управляющих байтов.

Управляющий байт кодирует состояние и H2

МЕХАНИЗМ

```
empty    = 1000 0000
deleted  = 1111 1110
full     = 0hhh hhhh

hhh hhhh = H2 (7 бит)
```

ПОЧЕМУ ТАК

- Управляющий байт одновременно кодирует состояние ячейки и короткий отпечаток хеша.
- Кодировка должна различать настоящую пустоту и метку удаления.

Почему метаданные отделены от полного ключа

МЕХАНИЗМ

```
сначала 8 маленьких управляющих
байтов
↓
отбросить почти все ячейки
↓
только потом трогать настоящие ключи
```

ПОЧЕМУ ТАК

- H2 — дешевый фильтр кандидатов; сравнение ключа необходимо для корректности.
- Проверять восемь ключей последовательно дорого.
- Вероятность случайного совпадения конкретной ячейки примерно:

Главный вопрос: восемь сравнений без восьми if

0	1	2	3	4	5	6	7
12	35	80	35	7F	FE	22	35

Ищем H2 = 35. Кандидаты — ячейки 1, 3 и 7.

control bytes → H2 filter → mask → сравнение реальных ключей

Одна группа Swiss Table = 8 слотов и 8 управляющих байтов.

Восемь управляющих байтов помещаются в uint64

МЕХАНИЗМ

```
ctrl bytes (little-endian)
[12][35][80][35][7F][FE][22][35]

g = 0x3522FE7F35803512
```

ПОЧЕМУ ТАК

- Восемь управляющих байтов помещаются в одно слово uint64.
- Операция над uint64 параллельно влияет на восемь байтовых позиций.

Сначала размножаем H2 по всем байтам

МЕХАНИЗМ

```
0x0101010101010101
× 0x35
————————————————————
0x3535353535353535
```

ПОЧЕМУ ТАК

- Отпечаток H2 размножается по всем восьми байтам управляющего слова.
- Теперь достаточно одного XOR со словом ctrl.

XOR превращает совпадение в нулевой байт

МЕХАНИЗМ

```
g = 0x3522FE7F35803512
H = 0x3535353535353535
──────────────────────── XOR
v = 0x0017CB4A00B50027

zero bytes: 1, 3, 7
```

ПОЧЕМУ ТАК

- Совпадающий H2 после XOR превращается в нулевой байт.
- Новая задача — найти нулевые байты в одном машинном слове.

Классический трюк поиска нулевых байтов

МЕХАНИЗМ

```
v = g XOR repeatedH2

((v - 0x0101010101010101)
  &^ v)
  & 0x8080808080808080
```

ПОЧЕМУ ТАК

- Нулевой байт после вычитания 1 превращается в FF, а его старший бит можно выделить маской.
- Переносимый алгоритм ищет все кандидаты пачкой; его маска может содержать редкое ложное совпадение.

Результат matchH2 на нашем примере

0	1	2	3	4	5	6	7
00	80	00	80	00	00	00	80

8-битные маски: зеленые байты содержат установленный старший бит.

```
mask = 0x8000000080008000
```

Одна группа Swiss Table = 8 слотов и 8 управляющих байтов.

Переносимый путь может дать редкое ложное совпадение

МЕХАНИЗМ

```
ячейка-кандидат
↓
настоящий ключ
↓
Equal(key, slotKey)
```

ПОЧЕМУ ТАК

- Ложное совпадение H2 означает лишнюю проверку ключа, но не ошибку результата.
- Потому что H2 никогда не является окончательным доказательством.
- Ложное совпадение означает только лишнее сравнение ключа.

На amd64 реальный путь еще интереснее

МЕХАНИЗМ

```
Переносимый путь
→ uint64 + AND/XOR/shift
→ бит 7 каждого байта

amd64 (Go 1.27)
→ compiler intrinsic / SIMD
→ плотная маска: 1 бит на slot
```

ПОЧЕМУ ТАК

- Алгоритмическая модель и фактический машинный код — разные уровни.
- Форму маски определяет реализация для архитектуры.

Как из маски получить первую ячейку

МЕХАНИЗМ

```
mask = ... 00 80 00 80 ...

TrailingZeros64(mask) = 15

index = 15 >> 3 = 1
```

ПОЧЕМУ ТАК

- Первый установленный бит — старший бит байта слота 1: 8 + 7 = 15.
- Сдвиг вправо на 3 переводит позицию бита в индекс байта.

Как взять следующего кандидата

МЕХАНИЗМ

```
mask != 0
  ↓
first = TrailingZeros64(mask) >> 3
  ↓
проверить slot first
  ↓
mask &= mask - 1
  ↓
повторить
```

ПОЧЕМУ ТАК

- x & (x-1) снимает младший установленный бит.
- Текущий рантайм скрывает это за bitset.removeFirst().

matchEmpty: найти конец последовательности пробирования

МЕХАНИЗМ

```
matchH2 → candidate keys

matchEmpty → конец цепочки?
├── найден empty → key отсутствует
└── empty нет → probe.next()
```

ПОЧЕМУ ТАК

- Настоящая пустая ячейка доказывает, что ключ не мог быть вставлен дальше.
- Deleted не равен empty: после удаления поиск должен продолжаться.

Как кодируется empty

МЕХАНИЗМ

```
empty   = 1000 0000
deleted = 1111 1110
full    = 0hhh hhhh

empty != deleted
```

ПОЧЕМУ ТАК

- У empty и deleted установлен старший бит, но следующие биты отличаются.
- matchEmpty обязан отличить конец поиска от метки удаления.

matchEmptyOrDeleted: для вставки задача другая

МЕХАНИЗМ

```
Lookup: deleted → continue

Insert: deleted → candidate slot

matchEmptyOrDeleted:
v & 0x8080808080808080
```

ПОЧЕМУ ТАК

- Для вставки удаленную ячейку можно рассматривать как свободное место.
- Но поиск существующего ключа все равно обязан пройти дальше.

Теперь полный поиск внутри одной группы

МЕХАНИЗМ

```
h2Hash
↓
matchH2
↓
маска кандидатов
↓
пока маска != 0:
    i = first(маска)
    сравнить настоящий ключ в ячейке i
    если равен → нашли
    иначе removeFirst
↓
matchEmpty
↓
empty есть?
├── да → ключ отсутствует
└── нет → пробировать следующую группу
```

ПОЧЕМУ ТАК

- Группа: matchH2 → Equal → matchEmpty → следующая группа или выход.
- Вот здесь Swiss Table становится алгоритмом, а не схемой.

Почему восемь ячеек — это не восемь шагов обычного линейного пробирования

МЕХАНИЗМ

```
одна компактная операция над ctrl word
↓
маска кандидатов
↓
обычно 0 или очень мало настоящих сравнений ключей
```

ПОЧЕМУ ТАК

- Одно сравнение control word заменяет серию полных сравнений ключей.
- Swiss Table использует метаданные группы:
- Наивная таблица с открытой адресацией:

Группа закончилась. Что дальше?

МЕХАНИЗМ

```
в этой группе ключа нет
+
мы еще не встретили конец цепочки пробирования
```

ПОЧЕМУ ТАК

- Нет совпадения H2 и нет empty — переходим к следующей группе.

Самое очевидное решение — линейное пробирование

МЕХАНИЗМ

```
g0
g0 + 1
g0 + 2
g0 + 3
...
```

ПОЧЕМУ ТАК

- Линейное пробирование формирует длинные плотные кластеры.
- Новые ключи, попавшие рядом, продолжают увеличивать тот же кластер.
- Swiss Table использует другую последовательность.

Текущая последовательность пробирования Go

МЕХАНИЗМ

```
index = 0
offset = H1 & mask

next():
    index++
    offset = (offset + index) & mask

шаги +1, +2, +3, +4, ...
```

ПОЧЕМУ ТАК

- Текущая последовательность использует треугольные числа по модулю 2^m.
- В отличие от линейного пробирования, последовательность быстрее уходит от первичного кластера.

Запишем последовательность математически

МЕХАНИЗМ

```
T(n) = n(n+1)/2

T0.. = 0, 1, 3, 6, 10, 15, 21 ...

slot(n) = (start + T(n)) & mask
```

ПОЧЕМУ ТАК

- Для степени двойки modulo вычисляется побитовой маской.
- Последовательность должна посетить каждую группу до первого повторения.

Пример: 16 групп

МЕХАНИЗМ

```
groupCount = 16, mask = 15
start = 5

T(n) mod 16:
0  1  3  6  10 15  5  12
4  13 7  2  14 11  9  8

visited:
5  6  8  11 15 4  10 1
9  2  12 7  3  0  14 13
```

ПОЧЕМУ ТАК

- Все 16 групп посещены ровно по одному разу.
- Это не случайная прогулка: порядок задан арифметикой треугольных чисел.

Почему треугольная последовательность покрывает 2^m позиций

МЕХАНИЗМ

```
T_i = i(i+1)/2
T_j = j(j+1)/2

T_j - T_i =
  (j-i)(i+j+1)/2

при i < j < 2^m:
один множитель четный,
другой нечетный
```

ПОЧЕМУ ТАК

- Предположение T_i ≡ T_j (mod 2^m) потребовало бы делимости произведения на 2^(m+1).
- В заданном диапазоне четный множитель не может обеспечить столько степеней двойки — противоречие.

Зачем нам эта математика в бэкенд-курсе

МЕХАНИЗМ

```
размер = степень двойки
+
треугольная последовательность пробирования
↓
гарантированно обходим все группы
```

ПОЧЕМУ ТАК

- Треугольная последовательность гарантирует обход всех групп размера 2^m.
- И при этом первые шаги достаточно быстро отходят от одного линейного кластера.

Высокая заполненность все равно ухудшает поиск

МЕХАНИЗМ

```
группа 1 заполнена
↓
группа 2 заполнена
↓
группа 3 заполнена
↓
...
```

ПОЧЕМУ ТАК

- Высокое заполнение увеличивает число посещенных групп и пробиваний.
- Растет вероятность реальных сравнений ключей.
- Получаем больше H2 ложные совпадения.

Текущий коэффициент заполнения

МЕХАНИЗМ

```
load factor = 7 / 8
            = 87.5%

больше заполнение
→ длиннее probing
→ больше обращений к памяти
```

ПОЧЕМУ ТАК

- Свободные ячейки — часть инварианта открытой адресации.
- Предел 7/8 — деталь Go 1.27, а не API.

Маленькая map: отдельный путь

МЕХАНИЗМ

```
маленькая map ≤ 8 записей

dirLen = 0
dirPtr → одна group

no directory lookup
no multi-group probing
```

ПОЧЕМУ ТАК

- Для одной группы работает упрощенный путь.
- При удалении не нужны tombstones для сохранения цепочки между группами.

Почему у маленькой map нет tombstones

МЕХАНИЗМ

```
нет переходов между группами
↓
нет необходимости сохранять доказательство «поиск должен идти
дальше»
↓
состояние deleted не нужно
```

ПОЧЕМУ ТАК

- В одной группе нет цепочки между группами — tombstones не нужны.
- Поэтому текущая оптимизация маленькой map может просто освободить ячейку без tombstone.
- Специальный маленький случай позволяет убрать часть общего алгоритма.

Полный поиск современной map

МЕХАНИЗМ

```
hash(key, seed)
    ↓
directoryIndex(hash)
    ↓
table / group
    ↓
matchH2(H2)
    ↓
Equal(key, candidate)
    ↓
matchEmpty? / probe.next()
```

ПОЧЕМУ ТАК

- Встроенная map проходит хеш, каталог, группу и проверку настоящего ключа.
- Каждый этап имеет свою цену: CPU, cache locality, comparison.

Вставка сложнее поискаа

МЕХАНИЗМ

```
найти ячейку
↓
обновить значение
```

ПОЧЕМУ ТАК

- Вставка обязана сначала доказать отсутствие ключа, а уже потом выбрать slot.
- И вот здесь deleted становится полезным.

Вставка запоминает первую метку удаления

МЕХАНИЗМ

```
probe groups
↓
первый deleted → запомнить slot
↓
продолжить поиск existing key
↓
нет existing key?
→ можно вставить в tombstone
```

ПОЧЕМУ ТАК

- Нельзя немедленно вставить в первую deleted-ячейку, не проверив продолжение цепочки.
- Deleted помогает сохранить lookup-инвариант.

growthLeft: таблица считает оставшийся бюджет роста

МЕХАНИЗМ

```
growthLeft
= remaining insertion budget

влияют:
  used slots
  deleted slots
  load factor
```

ПОЧЕМУ ТАК

- Метка удаления может ухудшать пробирование, даже если логический элемент удален.
- Очищение/перестройка восстанавливает бюджет роста.

Удаление: почему выбор между empty и deleted зависит от инварианта пробирования

МЕХАНИЗМ

стереть ключ/значение

ПОЧЕМУ ТАК

- Удаление меняет инвариант пробирования, не только содержимое элемента.
- Но если структура вокруг позволяет доказать, что цепочка не ломается, метка deleted может быть не нужна.
- После поиска удаляемого ключ нельзя механически написать:

Расти может одна таблица, а не вся map

МЕХАНИЗМ

```
Map
↓
каталог
↓
несколько независимых таблиц Swiss
Table
```

ПОЧЕМУ ТАК

- Рост локален: большая map разбита на отдельные ограниченные таблицы.
- Не каждый рост сразу означает разделение.

Сначала таблица может просто стать больше

МЕХАНИЗМ

```
capacity of local table:
8 → 16 → 32 → ... → 1024

пока помещается в 1024:
  grow local table
  reinsert entries
```

ПОЧЕМУ ТАК

- До предела таблица может удваивать собственную емкость.
- Предел 1024 slots — деталь текущей реализации.

Только большая таблица делится

МЕХАНИЗМ

```
capacity = 1024
хотим 2048
        ↓
не одна огромная table
        ↓
split → left + right
        ↓
еще 1 бит hash prefix
```

ПОЧЕМУ ТАК

- Когда локальная таблица достигла максимума, происходит разделение.
- После split каталог должен отличать две новые таблицы.

Что такое каталог

МЕХАНИЗМ

```
directory size = 2^globalDepth

globalDepth = 2:
00 → table A
01 → table A
10 → table B
11 → table C
```

ПОЧЕМУ ТАК

- Каталог — массив указателей на таблицы.
- Несколько индексов могут ссылаться на одну таблицу с меньшим localDepth.

globalDepth

МЕХАНИЗМ

```
directoryIndex(hash)
= hash >> (64 - globalDepth)

например globalDepth=3:
hash = 101101...
index = 101₂ = 5
```

ПОЧЕМУ ТАК

- globalDepth определяет, сколько старших бит хеша использует весь каталог.
- Количество directory entries равно 2^globalDepth.

localDepth

МЕХАНИЗМ

```
globalDepth = 2

00 ─┐
01 ─┴→ table A [localDepth=1]
10 ──→ table B [localDepth=2]
11 ──→ table C [localDepth=2]
```

ПОЧЕМУ ТАК

- Одна таблица может обслуживать несколько записей каталога.
- localDepth показывает, сколько бит хеш-префикса уже отличают именно эту таблицу.

Разделение таблицы руками

МЕХАНИЗМ

```
old prefix = 01
old localDepth = 2

split → localDepth = 3

010... → left
011... → right
```

ПОЧЕМУ ТАК

- Старую таблицу разделяют по следующему биту хеша.
- Для распределения записей нужен их хеш, который может вычисляться заново.

Почему при разделении нужно снова считать хеш

МЕХАНИЗМ

```
перебор записей
+
повторное хеширование ключей
+
новые вставки
```

ПОЧЕМУ ТАК

- При split нужно хешировать исходные ключи заново для следующего prefix-bit.
- Этого недостаточно, чтобы узнать старший бит префикса.
- Для разделения нужен следующий бит префикса его хеша.

Когда нужно удвоить каталог

МЕХАНИЗМ

```
if old.localDepth == globalDepth

dirLen → dirLen × 2
globalDepth++

[A, B]  →  [A, A, B, B]
```

ПОЧЕМУ ТАК

- Каталог удваивается, только когда текущей глубины недостаточно для split.
- Удвоение копирует ссылки, а не все ключи map.

Удвоение каталогаа не означает копирование всех ключей и значений

МЕХАНИЗМ

```
grow directory:
[A, B] → [A, A, B, B]

local split:
A → A0 + A1

other tables unchanged
```

ПОЧЕМУ ТАК

- Полная map не перехешируется при локальном разделении.
- Это распределяет стоимость роста по частям структуры.

Почему это уменьшает всплески задержки

МЕХАНИЗМ

```
переполнилась конкретная таблица
↓
рост / разделение только ее
↓
остальные таблицы не трогаем
```

ПОЧЕМУ ТАК

- Локальный split ограничивает скачки p99 относительно полного rehash N ключей.
- Но хвост задержки контролировать проще.
- Каталог + таблицы ограниченного размера:

Теперь откроем еще один черный ящик: хеширование ключа

МЕХАНИЗМ

ключ
↓
хеш

ПОЧЕМУ ТАК

- Hasher(key, seed) тоже расходует CPU; O(1) не отменяет стоимость вычисления хеша.

Цена хеширования зависит от типа ключа

МЕХАНИЗМ

```
map[uint64]V
→ 8 bytes key input

map[string]V
→ hash string contents

16 B != 16 KiB по цене хеша
```

ПОЧЕМУ ТАК

- O(1) по числу элементов map не означает O(1) по размеру ключа.
- Стоимость хеширования и сравнения ключа важна под нагрузкой.

Сравнение ключей тоже не бесплатно

МЕХАНИЗМ

```
type Key struct {
    Tenant string
    UserID string
    Region uint32
}
```

ПОЧЕМУ ТАК

- После совпадения H2 настоящий ключ сравнивается с помощью Key.Equal.
- Но не делает сравнение на равенство бесплатной.
- Метаданные Swiss Table минимизируют число таких сравнений.

Зерно хеша нужно не для красоты

МЕХАНИЗМ

Собственное зерно map делает размещение ключей менее предсказуемым.

ПОЧЕМУ ТАК

- Собственное зерно map делает размещение ключей менее предсказуемым.
- При этом точный алгоритм хеширования — деталь реализации и может различаться по типу/архитектуре/версии.
- Зерно конкретной map делает размещение ключей зависимым от экземпляра таблицы.

Под нагрузкой сложный ключ может стать заметен в профиле CPU

МЕХАНИЗМ

хеширование
+
сравнение ключей

ПОЧЕМУ ТАК

- При миллионах поисков стоимость hash/equality становится видна в CPU profile.
- Даже если Swiss пробирование почти всегда заканчивается на первой группе:
- могут стать заметной стоимостью по CPU.

sync.Map: конкурентное хеш-дерево

Почему атомарные чтения и локи отдельных узлов масштабируются иначе?

- **HashTrieMap**
- **4 бита / уровень**
- **atomic loads**
- **local node locks**

Исходник: 4.1 - Go structures deeper.md

sync.Map: обычная map уже не решает задачу целиком

МЕХАНИЗМ

```
type Cache struct {
    mu sync.RWMutex
    m  map[string]Value
}
```

ПОЧЕМУ ТАК

- map + RWMutex все еще стандартное решение при составных инвариантах.
- Но все ключи проходят через одна область синхронизации:
- Для некоторых сценариев доступа это становится лишней общей координацией.

Текущий sync.Map — буквально обертка над HashTrieMap

МЕХАНИЗМ

```
sync.Map
   ↓
internal/sync.HashTrieMap[any,any]
   ↓
path by 4-bit hash chunks
```

ПОЧЕМУ ТАК

- Историческая модель read/dirty/misses для Go 1.27 больше не описывает реализацию.
- Текущий механизм — конкурентное хеш-дерево.

Корень HashTrieMap

МЕХАНИЗМ

```
ключ
↓
хеш
↓
root
↓
часть хеша
↓
дочерний узел
↓
часть хеша
↓
дочерний узел
...
```

ПОЧЕМУ ТАК

- Путь HashTrieMap строится из последовательных четырехбитных фрагментов хеша.
- Но сколько бит используется на уровень?
- И функция хеширования ключа плюс зерно.

По четыре бита хеша на уровень

МЕХАНИЗМ

```
nChildrenLog2 = 4
nChildren = 16

64-bit hash
→ максимум 16 логических уровней
→ по 4 бита на уровень
```

ПОЧЕМУ ТАК

- Каждый indirect-узел выбирает одного из 16 потомков.
- Части хеша задают путь без общего линейного списка записей.

Узел indirect

МЕХАНИЗМ

```
indirect node
├── marker / dead
├── mutex
├── parent
└── children[16]
        └── atomic.Pointer
```

ПОЧЕМУ ТАК

- Чтение идет по атомарно опубликованным указателям.
- Структурное изменение требует локального mutex и повторной проверки.

Узел entry

МЕХАНИЗМ

```
entry node
├── marker
├── key
├── value
└── overflow chain
    (full hash collision)
```

ПОЧЕМУ ТАК

- Если все биты хеша одинаковы, trie не может разделить ключи дальше.
- Тогда используется overflow chain и сравнение настоящих ключей.

Load: чистое чтение идет по атомарным указателям

МЕХАНИЗМ

```
hashShift -= 4
idx = (hash >> hashShift) & 0xF

node = child[idx].Load()

nil → not found
entry → compare key
indirect → next level
```

ПОЧЕМУ ТАК

- Load следует по атомарным указателям без глобального read-lock.
- Цена — несколько зависимых pointer loads и сравнений.

Почему Load не берет общий mutex

МЕХАНИЗМ

глобальный лок корня

ПОЧЕМУ ТАК

- HashTrieMap Load использует атомики без глобального mutex; Store имеет локи.
- Он использует атомарную публикацию указателей на дочерние узлы.
- Структурные изменения используют локи внутренних узлов.

Store: сначала оптимистично читаем дерево

МЕХАНИЗМ

```
root
↓
4 бита
↓
дочерний узел
↓
...
```

ПОЧЕМУ ТАК

- Store сначала читает дерево оптимистично, затем синхронизирует изменение.
- Но между поиском и изменением другая горутина могла уже изменить узел.
- Путь вставки начинает похоже на Load:

Поэтому после локирования состояние проверяется снова

МЕХАНИЗМ

```
optimistic read
      ↓
найти candidate node
      ↓
lock(local indirect node)
      ↓
recheck state
  ├── unchanged → modify
  └── changed → retry
```

ПОЧЕМУ ТАК

- Пока мы ждали лок, другая горутина могла поменять структуру.
- Локализованный лок уменьшает конкуренцию независимых веток.

Если два ключа расходятся на следующих четырех битах

МЕХАНИЗМ

```
A: 1010 0011 ...
B: 1010 1110 ...

1-й уровень: child[1010]
2-й уровень:
  A → child[0011]
  B → child[1110]
```

ПОЧЕМУ ТАК

- Если ключи расходятся на следующем блоке бит, entry заменяется indirect-узлом.
- Trie растет в глубину только там, где этого требуют коллизии префикса.

Если полный хеш одинаковый

МЕХАНИЗМ

цепочка коллизий

ПОЧЕМУ ТАК

- При полной коллизии хеша trie не может разветвиться — нужна цепочка.
- HashTrieMap тоже не отменяет фундаментальную возможность коллизии хеша.
- И уже внутри нее настоящее сравнение ключей отличает A от B.

sync.Map не обещает согласованный снимок в Range

МЕХАНИЗМ

```
Range() ≠ consistent snapshot

concurrent Store/Delete
может менять видимый набор
во время обхода
```

ПОЧЕМУ ТАК

- Range не блокирует все записи ради одного согласованного снимка.
- Если нужен составной инвариант — потребуется другая синхронизация.

sync.Map против map + RWMutex: правильный вопрос

МЕХАНИЗМ

разные горутины в основном работают с разными ключами

ПОЧЕМУ ТАК

- sync.Map выигрывает не всегда: важен характер доступа и бизнес-инвариант.
- sync.Map особенно ориентирована на случаи:
- Какой у нас характер доступа и какой инвариант нужно защищать?

Под нагрузкой trie тоже имеет цену

МЕХАНИЗМ

```
несколько косвенных обращений через указатели
+
несколько атомарных загрузок
+
больше метаданных
+
локи при изменениях структуры
```

ПОЧЕМУ ТАК

- HashTrieMap платит за атомарные pointer loads, metadata и локальные блокировки.
- Поэтому нагрузка с одной горутиной или маленькой структурой не обязана выигрывать.
- Сложная конкурентная структура оправдывается тогда, когда она уменьшает более дорогую общую конкуренцию за ресурс.

Интерфейсы: точное представление

Где лежат Type, ITab и Data; сколько стоит проверка совместимости?

empty interface

ITab.Fun

TypeAssertCache

devirtualization

Исходник: 4.1 - Go structures deeper.md

Теперь интерфейсы: модель «тип + данные» становится недостаточной

МЕХАНИЗМ

```
интерфейс
=
динамический тип
+
данные
```

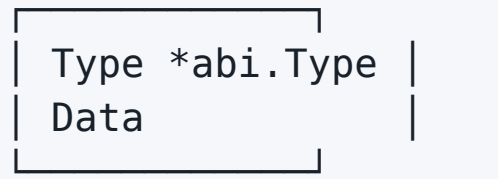
ПОЧЕМУ ТАК

- Точный interface ABI отвечает на вопрос, где тип, данные и указатели методов.
- Этого достаточно, чтобы объяснить typed nil, утверждение типа и динамический вызов.
- Для непустого интерфейса мы добавили ITab.

Пустой и непустой интерфейс устроены по-разному

МЕХАНИЗМ

any (empty interface)



interface with methods



ПОЧЕМУ ТАК

- Оба ABI-представления занимают два машинных слова.
- У any первое слово — Type, у непустого интерфейса — ITab.

Почему any не нужен ITab

МЕХАНИЗМ

```
тип интерфейса
+
конкретный тип
```

ПОЧЕМУ ТАК

- any хранит Type, а непустой interface — подготовленную связь ITab.
- Как именно конкретный тип реализует io.Writer?
- Нужна не просто информация о конкретном типе, а связь:

Что находится в ITab

МЕХАНИЗМ

```
ITab
├── Inter  → interface type
├── Type   → concrete type
├── Hash   → type hash
└── Fun[]  → method function PCs
```

ПОЧЕМУ ТАК

- ITab заранее соединяет конкретный тип и требуемый интерфейс.
- Метод не ищут по строковому имени при каждом вызове.

ITab.Fun подводит нас к динамическому вызову

МЕХАНИЗМ

```
w.Write(p)
↓
ITab
↓
элемент Fun
↓
адрес функции
↓
вызов
```

ПОЧЕМУ ТАК

- ITab.Fun превращает метод интерфейса в адрес конкретной функции.

Data не означает «обязательно указатель на объект в хипе»

МЕХАНИЗМ

где физически живет память этого значения?

ПОЧЕМУ ТАК

- Data не доказывает, что данные живут в хипе.
- Если значение попало в интерфейс, рантайм обязан создать отдельный объект в хипе, а Data указывает на него.
- Второе зависит от компилятора, эскейп-анализа и конкретного пути преобразования.

Прямое представление в интерфейсе

МЕХАНИЗМ

```
TFlagDirectIface
          ↓
Data word содержит
само pointer-like value

не любой объект size == 8 B
```

ПОЧЕМУ ТАК

- Прямая форма зависит от pointer layout, а не только от размера.
- int64 размером 8 байт не становится автоматически direct-interface value.

Почему int64 нельзя автоматически считать значением с прямым представлением

МЕХАНИЗМ

```
size <= machine word
→ прямое представление
```

ПОЧЕМУ ТАК

- sizeof(int64)==sizeof(pointer) не делает int64 direct interface автоматически.
- Для прямого представления важно, что тип является указательноподобным с точки зрения карты указателей.
- Раз размеры совпадают, положим биты int64 прямо в Data.

Косвенное представление

МЕХАНИЗМ

```
indirect interface:
Data → address of stored value

storage location:
stack? static? heap?

indirect != heap allocation
```

ПОЧЕМУ ТАК

- Размещение зависит от compiler/escape analysis.
- Преобразование к интерфейсу не означает обязательную аллокацию в хипе.

Конкретный тип → any

МЕХАНИЗМ

```
Type
→ дескриптор конкретного типа T

Data
→ представление значения v
```

ПОЧЕМУ ТАК

- Concrete → any: compiler знает Type и формирует Data representation.
- Вопрос только в том, как представить данные и требуется ли для этого отдельное хранилище.
- Компилятор уже знает статический тип v.

Конкретный тип → непустой интерфейс

МЕХАНИЗМ

```
(io.Writer, T)
↓
ITab
```

ПОЧЕМУ ТАК

- Concrete → interface с методами требует пары (interface type, concrete type).
- Если одна и та же пара (интерфейс, конкретный тип) встречается миллион раз, мы миллион раз сопоставляем методы?
- Для динамических случаев рантайм умеет искать или создавать подходящую запись.

itabTable: кеш уже построенных связей

МЕХАНИЗМ

```
getitab(interface, concrete)
        ↓
itabTable atomic read
        ↓
found?
├── yes → ready ITab
└── no  → locked slow path
```

ПОЧЕМУ ТАК

- Пара interface+concrete type переиспользуется через кеш.
- Глобальный lock нужен только при построении/публикации новой связи.

Если ITab еще нет

МЕХАНИЗМ

```
miss itabTable
     ↓
lock(itabLock)
     ↓
recheck (race with creator?)
     ↓
create + itabInit
     ↓
publish to itabTable
```

ПОЧЕМУ ТАК

- Повторная проверка под локом предотвращает дублирование ITab.
- Сопоставление методов выполняется при создании, не на каждый динамический вызов.

itabInit: как проверить реализацию интерфейса

МЕХАНИЗМ

```
interface methods (sorted)
+ concrete methods (sorted)
            ↓
merge-like scan
            ↓
O(ni + nt)
not O(ni × nt)
```

ПОЧЕМУ ТАК

- Нужно проверить соответствие методов интерфейса конкретному типу.
- Упорядоченность наборов методов позволяет проходить их почти как при слиянии.

После itabInit вызовы уже не ищут методы по имени

МЕХАНИЗМ

```
Fun[0]
Fun[1]
...
```

ПОЧЕМУ ТАК

- После itabInit поиск метода по строке не нужен.
- заполняются адресами функций, соответствующих методам интерфейса.
- не нужен поиск строки "Write" в какой-то таблице методов.

Интерфейс → интерфейс: появляется новая проверка

МЕХАНИЗМ

```
этот тип
↓
реализует io.Writer?
```

ПОЧЕМУ ТАК

- Interface → interface требует проверить реализуемость целевого интерфейса.
- Нужно получить подходящий ITab для пары:
- То есть задача уже не сводится к сравнению одного Type с другим.

Утверждение к конкретному типу проще

МЕХАНИЗМ

```
извлечь данные
↓
ok = true
```

ПОЧЕМУ ТАК

- Assertion к concrete type может сводиться к сравнению Type pointers.
- В однорезультатной форме отрицательный результат приводит к panic.
- Для пустого интерфейса полезная модель:

Утверждение к интерфейсу — это поиск совместимости

МЕХАНИЗМ

```
ITab(dynamicConcrete, io.Writer)
```

ПОЧЕМУ ТАК

- Assertion к interface требует ITab(dynamic concrete, target interface).
- Именно поэтому утверждение к конкретному типу и утверждение к интерфейсу — разные механические задачи.
- io.Writer может реализовываться множеством разных типов.

TypeAssertCache: горячий повторный путь

МЕХАНИЗМ

```
TypeAssertCache
key = source dynamic Type
value = target ITab / nil

cache hit → cheap result
cache miss → getitab slow path
```

ПОЧЕМУ ТАК

- Кеш может запомнить и отрицательный результат assertion с ok.
- Расширение кеша редкое и вероятностное — не платим за холодный путь.

Почему кеш не перестраивается после каждого промаха

МЕХАНИЗМ

```
Кеш утверждений типа обновляется не на каждый промах.
```

ПОЧЕМУ ТАК

- Кеш утверждений типа обновляется не на каждый промах.
- Если конкретное утверждение выполнилось один раз, эта работа не окупится.
- Не платить дорогую цену оптимизации редкого пути.

type switch: та же задача сразу для нескольких интерфейсов

МЕХАНИЗМ

```
type switch
case io.Reader
case io.Writer
case fmt.Stringer

InterfaceSwitchCache
Type → case index + ITab
```

ПОЧЕМУ ТАК

- Несколько интерфейсных веток проверяют один dynamic concrete type.
- Кеш сохраняет индекс ветки и соответствующий ITab.

Что происходит при промахе type switch-кеша

МЕХАНИЗМ

```
первый дорогой путь
↓
накопили информацию
↓
повторный горячий путь дешевле
```

ПОЧЕМУ ТАК

- При промахе InterfaceSwitchCache последовательно проверяются target interfaces.
- Если кеш не помог, рантайм может последовательно проверять интерфейсные case:
- Когда подходящая ветка найдена, результат может позже попасть в кеш.

Динамический вызов метода через ITab

МЕХАНИЗМ

```
w.Write(p)
     ↓
w.ITab
     ↓
ITab.Fun[slot]
     ↓
indirect function call
     ↓
concrete receiver Data
```

ПОЧЕМУ ТАК

- Метод уже размещен в конкретном слоте таблицы, не ищется по имени.
- Но компилятор может вообще устранить динамический dispatch.

Почему косвенный вызов может быть дороже прямого

МЕХАНИЗМ

```
var w io.Writer = buf
w.Write(p)
```

ПОЧЕМУ ТАК

- Косвенный вызов может мешать инлайнингу и последующим оптимизациям.
- Но здесь нельзя рассуждать только по синтаксису.
- Это влияет не столько на цену одной дополнительной загрузки, сколько на возможности оптимизатора:

Девиртуализация: интерфейсный вызов может исчезнуть

МЕХАНИЗМ

```
interface call
      ↓
concrete receiver known?
├── yes → direct call
│         ↓
│       inlining possible
└── no  → ITab dispatch
```

ПОЧЕМУ ТАК

- Девиртуализация — решение компилятора, не изменение ABI-интерфейса.
- Выигрыш часто происходит из-за открывшегося инлайнинга.

PGO добавляет еще один вариант девиртуализации

МЕХАНИЗМ

```
hot type from profile?
        ↓
guarded direct call
├── hot type → fast direct
└── other → generic fallback
```

ПОЧЕМУ ТАК

- PGO позволяет оптимизировать тип, доминирующий в горячей статистике.
- Конкретное решение зависит от профиля и компилятора.

Собираем четыре конструкции рядом

slice

growslice
nextslicecap
roundupsize
memmove

map

hash → directory
matchH2 → probing
split

sync.Map

HashTrieMap
atomic read
node lock

interface

Type / ITab
Data
type assertion / call

Один синтаксический язык — четыре совершенно разных механизма исполнения.

Нагрузка, диагностика и границы гарантий

Механизм → ограниченный ресурс → симптом → измерение

CPU	alloc_space	benchmem	SSA / assembly

Исходник: 4.1 - Go structures deeper.md

Что происходит под нагрузкой: срез

МЕХАНИЗМ

```
частые переходы через границу cap
↓
growslice
↓
аллокация + memmove
```

ПОЧЕМУ ТАК

- Частый growslice = поток аллокаций + memmove + давление на GC.
- Оптимизируем не append вообще, а частые и крупные операции роста.

Что происходит под нагрузкой: map

МЕХАНИЗМ

```
дорогой хеш ключа
длинное сравнение ключей
высокий коэффициент заполнения
частый рост / перехеширование
большой рабочий набор с плохой локальностью
```

ПОЧЕМУ ТАК

- Стоимость map: хеш + probing + equality + обращения к кешу CPU.
- Нужно понять, какая именно часть пути стала дорогой.
- Причины высокой цены могут быть разными:

Что происходит под нагрузкой: sync.Map

МЕХАНИЗМ

```
конкуренция за один общий лок ↓
```

ПОЧЕМУ ТАК

- Для sync.Map важно, уменьшает ли локализация конкуренцию по сравнению с общим локом.
- Поэтому sync.Map — специализированная структура, а не «ускоренная map».

Что происходит под нагрузкой: интерфейсы

МЕХАНИЗМ

```
10 000 000 операций/с
```

ПОЧЕМУ ТАК

- Интерфейсы стоят по-разному: conversion, type assertion, dispatch и возможные аллокации.
- Одна операция преобразования или одно утверждение типа обычно почти неинтересны.
- аллокации, если для представления требуется отдельная память;

Как диагностировать

СИМПТОМ	ИНСТРУМЕНТ	ЧТО ДОКАЗЫВАЕМ
Срезы	benchmem, alloc_space	Частота / объем growslice
map	CPU profile	Hasher, Equal, probing, рост
sync.Map	параллельный benchmark	Выигрыш локализованных изменений
Интерфейс: allocation	benchmem, escape	Где появляется storage
Интерфейс: dispatch	CPU → SSA → assembly	Остался ли косвенный вызов
Go upgrade	benchmark + profile	Изменились ли internals

Диагностируем конкретный скрытый механизм, а не синтаксис конструкции.

Что является гарантией языка и API

МЕХАНИЗМ

```
ЯЗЫК / API

append / len / cap
map semantics
interface + assertion
sync.Map public contract
```

ПОЧЕМУ ТАК

- Внешняя семантика остается стабильнее конкретного алгоритма рантайма.
- Прикладной код должен опираться на спецификацию и публичный API.

Что относится к текущей реализации Go 1.27.x

МЕХАНИЗМ

```
GO 1.27.x  (implementation)

threshold=256; growth formula
8-slot group; H2 bit tricks
maxTableCapacity=1024
HashTrieMap: 16 children
ITab / TypeAssertCache
```

ПОЧЕМУ ТАК

- Пороги, битовые приемы, названия структур и кеши могут меняться.
- Их изучаем для диагностики, но не закрепляем в бизнес-логике.

Разрушаем последние плохие упрощения

Срез всегда ×2	nextslicecap + округление размера
Swiss Table сравнивает ключи сразу	Сначала H2, потом Equal для кандидатов
sync.Map полностью lock-free	Load через атомики; изменения под локами
Interface всегда heap + indirect call	Storage зависит от escape; вызов может devirtualize

Граница «учебная модель → реализация → инженерный вывод» должна оставаться явной.

Полная цепочка одного append

01 append

↓

02 cap хватает?

↓

03 growslice

↓

04 nextslicecap

05 roundupsize

↓

06 mallocgc

↓

07 memmove

↓

08 записать новое

append имеет встроенную проверку cap и медленный путь через growslice.

Полная цепочка одного поиска в map

01 hash(key, seed)

↓

02 directory / small map

↓

03 group

↓

04 matchH2

05 Equal

↓

06 matchEmpty

↓

07 probe.next()

Полный lookup: hash → directory → group → matchH2 → Equal → probe.

Полная цепочка роста map

01 growthLeft == 0

↓

02 grow local table?

↓

03 local doubling

↓

04 или split

05 localDepth++

↓

06 globalDepth?

↓

07 directory ×2

Рост map: локальное удвоение либо split таблицы с возможным удвоением directory.

Полная цепочка sync.Map.Load

01 hash(key, seed)

↓

02 root.Load()

↓

03 4 bits → index

↓

04 child.Load()

05 entry → Equal

↓

06 indirect → repeat

↓

07 collision overflow

sync.Map.Load: атомарный root → 4-битные индексы → entry/indirect.

Полная цепочка утверждения типа к интерфейсу

01 x.(io.Writer)

↓

02 TypeAssertCache

↓

03 cache hit?

↓

04 getitab

05 itabTable

↓

06 itabInit + Fun[]

↓

07 result ITab / false

Assertion к интерфейсу: TypeAssertCache → getitab → готовый ITab.

Что важно запомнить

Срез	nextslicecap → roundupsize → mallocgc → memmove
Map	H1/H2 → group mask → probe → split
sync.Map	HashTrieMap → 4 бита/уровень → atomics + local locks
Интерфейс	Type/ITab + Data → conversion / assertion / dispatch

Весь материал — детали реализации Go 1.27.x; контракт языка выше конкретных порогов и структур.

Финальная инженерная модель

SLICE	MAP	SYNC.MAP	INTERFACE
append	hash	HashTrieMap	Type / ITab
nextslicecap	matchH2	4 bits / level	Data
roundupsize	probeSeq	atomic Load	TypeAssertCache
memmove	directory/split	node mutex	method dispatch

Go-код → алгоритм → память / CPU / синхронизация → нагрузка → измерение.

Что можно доказать руками после лекции

growslice	append → изменение cap → benchmem / alloc_space
matchH2	0x3522FE7F35803512 → H2=35 → slots 1,3,7
probeSeq	+1,+2,+3... mod 2^m → покрытие всех групп

Следующий слой — 4.3: какой SSA и машинный код оставил компилятор для этих операций.